

The U-Shaped Challenge of Language Grounding: How Instruction Granularity Induces VLA Shortcuts

Anonymous ACL submission

Abstract

Instruction granularity remains an uncontrolled variable in language-guided embodied AI, leaving its impact unknown. We propose an agent-centric formalization of granularity as rule width (w), a cross-task metric adapted from width-based planning. By measuring the peak concurrent state-feature dependencies an agent must resolve, w provides a rigorous measure of the extent to which an instruction reduces the decision-making burden required to map language into actions. We also introduce MINI-BEHAVIOR-GRAN, a controlled benchmark systematically varying w across 20 tasks while holding perception and control constant. Operationalizing granularity as rule width (w) reveals a non-monotonic, U-shaped performance curve: while fine-grained instructions ($w \leq 1$) ease grounding, coarse ones ($w \geq 2$) induce “language-ignoring” shortcuts driven by high instruction predictability ($P(l|v)$). We further uncover a generalization asymmetry, where coarse-trained models transfer to fine instructions but not vice-versa, and show that mixed-granularity training is essential for robust language grounding.

1 Introduction

Despite the rapid progress in language-guided embodied AI, the community has yet to establish a consensus on a unified metric to quantify the granularity of instructions during language grounding. In this domain, language serves a specific functional role: it provides structural information to decompose complex tasks and reduce the search space of the underlying sequential decision-making process. Consequently, *instruction granularity* should not be conflated with superficial syntactic properties, such as sentence length or the number of entities mentioned; rather, it must rigorously reflect the *decision-making burden* that remains for the agent to resolve internally after grounding the language into the action space.

Existing research conceptualizes instruction granularity through two dominant lenses, both of which struggle to provide a consistent measure of the decision-making burden required for grounding. Studies in Natural Language Processing (NLP) typically operationalize granularity via surface-form properties, such as textual specificity or discourse structure (e.g., number of entities mentioned) (Dong and Lapata, 2018; Huang et al., 2025b). While these effectively measure syntactic complexity, they adopt a *text-centric* rather than *agent-centric* view. Consequently, they cannot quantify how much an instruction reduces an embodied agent’s *decision-making burden*.

From an *agent-centric* perspective, the essential function of instruction is to decompose complex tasks into tractable sub-problems, thereby reducing the planning burden (Drexler et al., 2022). Recent work in LLM-based agents thus treats granularity as *decomposition depth*—the extent to which an instruction enumerates subgoals versus stating only the final objective (Yang et al., 2024; Myers et al., 2024; Shi et al., 2025; Zhong et al., 2025). While this aligns with our view that *instruction granularity* modulates planning demand, decomposition depth remains an unstable, task-dependent proxy. It conflates instructional guidance with the task’s inherent structure: complex tasks with interleaved subgoals may resist breakdown (e.g., the Sussman Anomaly (Sussman, 1975)), while trivial ones can be artificially elongated.

To operationalize this, we draw inspiration from the concept of policy sketches (Drexler et al., 2024) and formalize instruction granularity as rule width (w) (Bonet et al., 2019). We argue that w provides a consistent, cross-task proxy for the decision-making burden that an instruction imposes on an embodied agent. To this end, the contributions of our paper are:

(1) **Width as a Formalism for Instruction Granularity:** We formalize instruction granularity

as *rule width* (w), adapting a concept from width-based planning (Bonet et al., 2019; Drexler et al., 2024) that measures the peak number of concurrent state variables (features Φ) that must be tracked to fulfill the instructions. We argue w measures the *intrinsic decision complexity* of a task-instruction pair, regardless of whether the agent is symbolic or neural (§ 3).

(2) A Controlled Benchmark: We present MINI-BEHAVIOR-GRAN, the first testbed to isolate instruction granularity from confounding factors like task horizon and perception. It enables the first rigorous study of how instruction granularity affects VLA performance (§ 4).

(3) The U-Shaped Trend and Width Cap: We identify a non-monotonic U-shaped relationship where performance peaks at the Fine ($w = 0$) and Coarse ($w \geq 2$) extremes. We also identify a performance collapse at high width ($w \geq 4$): agents fail to fulfill instructions requiring maintaining concurrent features ≥ 4 , revealing a sharp language grounding boundary of existing VLA models (§ 5).

(4) Language-Ignoring Shortcuts: We diagnose the performance rebound at $w \geq 2$ as “Language Ignoring,” where agents bypass linguistic grounding to exploit visual shortcuts due to high instruction predictability ($P(l|v)$). We show that *Axial Training* (mixed-granularity) mitigates this failure mode. Additionally, we identify an *asymmetric generalization* pattern: agents trained on coarse instructions successfully transfer to fine-grained ones, but not vice versa.

2 Related Work

Instruction Abstraction and Granularity. In this work, *instruction granularity* refers to the extent to which state and action constraints are explicitly specified versus left for the agent to infer when grounding language into actions. This is distinct from, yet often conflated with the concept of *linguistic abstraction*. The term *abstraction* is viewed as *representational compression*—using a compact description to stand for multiple instances or executions. This definition is common in prior work of computational thinking (Wing, 2010; Lachmy et al., 2022; Zheng et al., 2024). Importantly, granularity can vary independently of abstraction: “tidy the room” is coarse but non-abstract (it leaves the exact required actions underspecified, yet it lacks representational compression.), whereas “repeat wiping until clean” is relatively specific yet abstract (it

prescribes an action but uses a loop condition to compress multiple executions).

While not framed as the term “granularity”, we acknowledge that a rich body of work has focused on augmenting language specificity and its impact, for example through coarse-to-fine semantic parsing (Dong and Lapata, 2018; Li et al., 2020). Other work generates compositional instructions via constraint dropout and sub-task recombination (Huang et al., 2025b), or construct counterfactual instructions to augment data diversity (Glossop et al., 2025).

In embodied instruction following, varying instruction detail is often handled via a simple two-part decomposition: high-level plans are converted into sequences of low-level steps before being grounded into actions (Yang et al., 2024; Shi et al., 2025). Most recent VLA works, however, operate with static language instructions (Ma et al., 2025; Li et al., 2025). Some models like InstructVLA and ThinkAct explores having chain-of-thought reasoning steps before action decoding (Yang et al., 2025; Huang et al., 2025a). These hierarchical definitions remain task-specific and qualitative, lacking a unified, cross-task metric to establish shared granularity levels across tasks. Consequently, quantitatively controlled studies on the formal dynamics of instruction granularity remain scarce in the embodied AI domain.

Language Sensitivity vs. Ignoring in VLA. Recent VLA research highlights a dichotomy between fragile language sensitivity (Akula et al., 2022) and systematic language-ignoring (Xing et al., 2025). We reconcile these via rule width (w): low w (Fine) involves explicit feature constraints and low predictability ($P(l|v)$), which enforces the agent to rely on language cues. Conversely, $w \geq 2$ (Coarse) triggers ignoring via visual shortcuts as $P(l|v)$ increases. Crucially, we identify a hard capacity cap at $w \geq 4$, where agents fail to track concurrent state-features regardless of granularity, revealing a fundamental grounding boundary in current architectures.

3 Rule Width as Instruction Granularity

3.1 Running Example

We first illustrate *planning width* computation using a running example. Table 1 shows the desired state progressions of a task where two shoes are cleaned using a rag, and the rag is placed on the floor afterwards. The key **environment dynamics**

State Progression	Concurrent Features Tracked (Sequence of Tuples)	Width
Initial State	$[\langle \rangle]$	0
Navigate to rag	$[\langle p_r = X \rangle, \dots, \langle p_r = 0 \rangle]$	1
Pick up rag	$[\langle p_r = 0 \rangle, \langle H_r \rangle]$	1
Navigate to sink	$[\langle H_r, p_s = X \rangle, \dots, \langle H_r, p_s = 0 \rangle]$	2
Turn on tap (sink)	$[\langle H_r, p_s = 0 \rangle, \langle T_s \rangle]$	2
Soak rag	$[\langle H_r, T_s, p_s = 0 \rangle, \langle S_r \rangle]$	3
Navigate to Shoe 1	$[\langle H_r, S_r, p_{f1} \downarrow \rangle, \dots]$	3
Clean Shoe 1	$[\langle H_r, S_r, p_{f1} = 0 \rangle, \langle C_1 \rangle]$	3
Navigate to Shoe 2	$[\langle H_r, S_r, C_1, p_{f2} \downarrow \rangle, \dots]$	4
Clean Shoe 2	$[\langle H_r, S_r, C_1, p_{f2} = 0 \rangle, \langle C_1, C_2 \rangle]$	4
Put rag on floor	$[\langle C_1, C_2, H_r \rangle, \dots, \langle C_1, C_2, onfloor_r \rangle]$	3

Table 1: State progression and width for the “Cleaning k Shoes” ($k = 2$). Width scales as $w = k + 2$.

are: (1) the rag must be soaked before it can clean shoes, and (2) the tap (sink) must be turned on to soak the rag. We represent states using a set of task-relevant feature variables (Φ): (i) boolean features H_r (holding rag), T_s (tap (sink) is turned on), S_r (rag is soaked), C_i (shoe i is cleaned), $onfloor_r$ (rag is on the floor); and (ii) spatial features p_r, p_s, p_f (distance to the rag, sink, and nearest uncleaned shoe, respectively).

First, approaching the rag is a navigation-only segment: the desired state progression is to decrease the distance feature p_r until $p_r = 0$. Thus, the width is 1 as only a single feature needs to be tracked. In this environment dynamics, $p_r = 0$ enables the state transition to H_r . Thus, the rag can be obtained at $p_r = 0$ and the width remains 1. Subsequently, H_r must be maintained while progressing to next states, resulting in a width of 2. The rag becoming soaked requires simultaneous maintenance of three features, i.e., at the sink ($p_s = 0$), holding the rag (H_r), and having the tap (sink) turned on (T_s), which yields a width of 3. After soaking, C_1 can be obtained by tracking H_r , S_r , and p_f , giving width 3. The subtlety arises when cleaning the second shoe: one has to track the same features as before, plus maintaining the cleanliness of the first shoe (C_1), which increases the width to 4.

Why no action model? Agents’ action sets constrain how state transitions are executed, but *width* focuses on state-level feature dependencies induced by the environment dynamics. Different agents (e.g., a rover vs. a drone) may have different action spaces to achieve the same feature change, so emphasizing state progressions promotes generalization of width-based concepts across different embodiments.

The impact of *instruction granularity* on VLA behavior remains underexplored largely because

the field lacks a **formal, quantifiable, cross-task** measure of granularity. The most common heuristic, *decomposition depth* (Myers et al., 2024; Shi et al., 2025), is unstable as discussed in § 1. To bridge this gap, we formalize *instruction granularity* as *planning demand*—the internal search required to ground an instruction into an executable plan. This perspective captures the intuitive difference between fine- and coarse-grained instructions: fine-grained instructions provide a “breadcrumb trail” that narrows the search, whereas coarse instructions leave a much larger solution space to be explored to reach a state that fulfills the instruction.

We operationalize planning demand using rule width (w), framing natural-language instructions as policy sketches (Drexler et al., 2024). A policy sketch is a set of “if-then” rules that decompose a complex task into a simpler sub-problems. Under this framing, the granularity of an instruction corresponds to the structural guidance it provides: fine-grained instructions create sub-problems with minimal width, while coarse instructions leave a larger “grounding gap” for the agent to resolve internally.

3.2 Symbolic Task Representation

While VLAs operate on raw multimodal observations, we adopt a symbolic, relational state abstraction. Many embodied benchmarks provide such structured representations—object properties and relations—that can be expressed as *fluents* to form symbolic state or goal conditions (Shridhar et al., 2020; Li et al., 2024). Formally, we consider a classical planning problem $P = \langle \mathcal{F}, \mathcal{A}, I, G \rangle$, where $\mathcal{F}$ is a set of fluents, $\mathcal{A}$ a set of actions, I the initial state, and G the goal.

To bridge raw states and high-level instructions, we define a set of **features** $\Phi = \{\phi_1, \dots, \phi_n\}$, where ϕ_i is a Boolean or integer-valued function over state space S . These features serve as domain-level abstractions that capture task-relevant aspects of the environment. Thus, each s is associated with a **feature valuation vector** $\phi(s) = (\phi_1(s), \dots, \phi_n(s))$.

3.3 Instructions as Rule Sets $\mathcal{R}$

To study instruction granularity in a controlled manner, we formalize each instruction a symbolic interpretation as a set of “if-then” rules $\mathcal{R} = \{r_1, \dots, r_m\}$ defined over the feature space Φ . This rule-based representation does not aim to capture the full expressiveness of natural language;

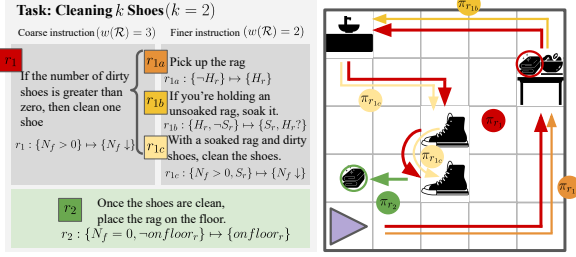


Figure 1: (Left) factoring r_1 into r_{1a} - r_{1c} reduces the planning complexity that the agent must handle internally. More explicit linguistic guidance makes the mapping to action easier to learn. (Right) the long-horizon trajectory for the coarse rule (π_{r_1} , red) is broken into shorter segments by the finer-grained instructions.

instead, it distills the constraint/goal structure that governs the computational cost incurred by a symbolic search algorithm (i.e., planning demand). Specifically, each r_i is of the form $C_{r_i} \mapsto E_{r_i}$:

- C_{r_i} (the *condition*) is a conjunction of feature constraints. For a Boolean feature p , a constraint is p or $\neg p$; for a numerical feature n , it is $n = 0$ or $n > 0$.
- E_{r_i} (the *effect*) is a conjunction of feature changes. For a Boolean feature p , an effect is p , $\neg p$, or $p?$ (allowing any change); for a numerical feature n , it is $n \downarrow$ (decrease), $n \uparrow$ (increase), or $n?$ (any change).

These forms capture the core semantics of the instructions we study. A rule r_i is **fulfilled** by a state sequence $(s, \dots, s')$ if $\phi(s) \models C_i$ and the pair $(\phi(s), \phi(s'))$ satisfies the changes in E_i (e.g., $\forall (n \downarrow) \in E_{r_i}, \phi_n(s') < \phi_n(s)$) while keeping unmentioned features constant (i.e., $\phi_j(s') = \phi_j(s)$ for $j \notin E_i$). To ensure that the agent's performance strictly reflects its grounding ability rather than the unsolvability of the task, we guarantee that all instruction rule sets $\mathcal{R}$ are executable and dead-end-free. Formally, this is achieved by ensuring the rules are well-formed (i.e., they terminate at the goal) and satisfy the soundness and closure properties (Bonet and Geffner, 2021).

3.4 Rule Width as a Granularity Measure

The width $w(r_i)$ of a rule r_i represents the number of state variables (features) that must be tracked concurrently to fulfill the transition. Formally, for a rule to have width k , the agent must coordinate at most k features from Φ to reach the effect E_i from a state satisfying C_i (Lipovetzky and Geffner, 2012).

We define the granularity of an instruction $\mathcal{R}$ by its width-based planning bottleneck: the maximum width among its constituent rules:

$$w(\mathcal{R}) = \max_{r_i \in \mathcal{R}} w(r_i). \quad (1)$$

We continue the running example to demonstrate two instruction sets, each representing a different width level, with two variants visualized in Figure 1. We also append a **counting feature** in Φ to track the number of uncleaned shoes, denoted as N_f .

Coarse-Grained Instruction ($w(\mathcal{R}) = 3$) This instruction states only the high-level rules and their desired ordering, leaving the internal structure of each sub-task implicit:

- $r_1 : \{N_f > 0\} \mapsto \{N_f \downarrow\}$ (clean a shoe; $w = 3$)
- $r_2 : \{N_f = 0, \neg on_floor_r\} \mapsto \{on_floor_r\}$ (put the rag on the floor after cleaning shoes; $w = 1$)

Middle-Grained Instruction ($w(\mathcal{R}) = 2$) This variant decomposes r_1 to explicitly state the *soaking* process:

- $r_{1a} : \{\neg H_r\} \mapsto \{H_r\}$ (pick up the rag; $w = 1$)
- $r_{1b} : \{H_r, \neg S_r\} \mapsto \{S_r, H_r\}$ (soak the rag)
- $r_{1c} : \{N_f > 0, S_r\} \mapsto \{N_f \downarrow\}$ (clean; $w = 2$)
- $r_2 : \{N_f = 0, \neg on_floor_r\} \mapsto \{on_floor_r\}$ (put the rag on the floor; $w = 1$)

This rule set is well-formed: it is goal-terminating via r_2 , and the rules are sequentially reachable, meaning the successor state of any fulfilled rule (that is not the goal) also meets the condition of at least one rule in $\mathcal{R}$. The set is also robust to perturbations: if an agent drops the rag after soaking it ($\neg H_r$), the rule set remains applicable by reverting to r_{1a} to reacquire the rag before proceeding to the cleaning task (r_{1c}).

Width as a Planning Proxy. From an algorithmic perspective, rule width $w(r)$ is the minimum conjunction size k required to resolve an instruction via novelty-driven Breadth-First Search (BFS). By pruning states that fail to achieve a new feature conjunction of size $\leq k$, the search complexity is bounded by $O(|\Phi|^k)$ (Lipovetzky and Geffner, 2017). In task r_{1b} (soaking a rag), an isolated transition $\{\neg S_r\} \mapsto \{S_r\}$ requires concurrently coordinating three features: holding the rag (H_r), reaching the sink ($p_s = 0$), and toggling the tap

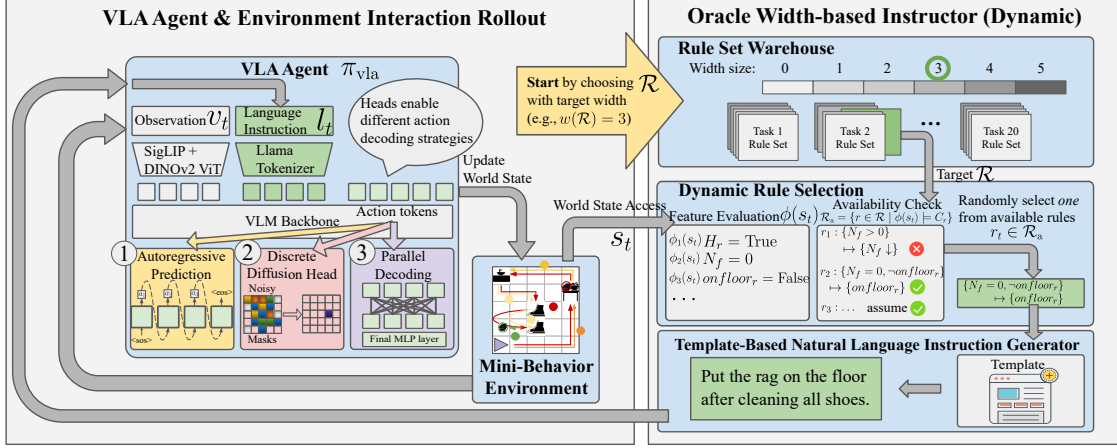


Figure 2: MINI-BEHAVIOR-GRAN Experimental Pipeline. We evaluate the impact of instruction granularity across three VLA action-decoding strategies. During inference time, an Oracle Instructor evaluates the current world state to decide available rule set $\mathcal{R}_a$ and active rule r_t . A rule-based natural language instruction l_t is generated accordingly. The process iterates until completion or max steps.

(T_s). However, since r_{1b} follows the establishment of H_r in r_{1a} , BFS treats H_r as a maintained feature. Since undoing this precondition yields longer, non-optimal paths that are effectively ignored by BFS, active coordination reduces to the conjunction $\langle T_s, p_s = 0 \rangle$, yielding $w(r_{1b}) = 2$. While VLAs are non-symbolic, we show that w serves as a rigorous proxy for the inherent planning burden imposed by an instruction, as evidenced by the performance trends observed in § 5.

To communicate these rules to VLA agents, we translate each rule into a natural-language sentence via a template-based generator. These templates verbalize the features within each rule; e.g., $\{H_r, \neg S_r\} \mapsto \{S_r\}$ is realized as “If you are holding the rag but the rag is not yet soaked, soak the rag.” This construction preserves the granularity differences defined by the rule width. Additional language prompt examples are provided in § F.

4 Mini-BEHAVIOR-Gran Benchmark

To disentangle the influence of instruction granularity in VLA performance, we seek an environment that (1) retains long-horizon and heterogeneous activities so that instruction guidance meaningfully modulates the planning demand, while (2) simplifies perception and low-level control to avoid confounding factors. As such, we use MINI-BEHAVIOR (Jin et al., 2023) as our testbed. Detailed justification for our choice over other environments is provided in § B.

We introduce MINI-BEHAVIOR-GRAN, the

first VLA benchmark specifically designed to enable a controlled study and formal quantification of instruction granularity via *rule width*. Specifically, we define a shared, domain-level feature set Φ that captures task-relevant attributes across all 20 complex household tasks on top of the symbolic state abstraction from the original benchmark. For each task, we design rule sets with varying widths (up to six levels). We verify that all rule sets are well-formed and solvable. We utilize an Oracle Instructor to deliver instructions during inference. This ensures that any observed performance drops are strictly attributable to the model’s inability to ground specific granularity levels, rather than stochastic variations in instruction quality or applicability that would confound our cross-granularity analysis (see Figure 2 right panel). At each time t , it evaluates the current feature vector $\phi(s_t)$, identifies the set of available rules $\mathcal{R}_a = \{r \in \mathcal{R} \mid \phi(s_t) \models C_r\}$. We sample an active rule $r_t \in \mathcal{R}_a$ and it is then processed by a template-based translator to generate a natural language instruction l_t , to match the VLA model’s input format. l_t is paired with the current observation v_t and fed into the VLA model to predict action chunk $(a_t, \dots, a_{t+k}) \sim \pi_{\text{vla}}(\cdot \mid v_t, l_t)$.

5 Experiments

5.1 Experimental Setup

Task Suite & Training. We use MINI-BEHAVIOR-GRAN, which contains 20 complex tasks sharing the same feature set Φ . We further classify instruction granularity into three *relative*

classes of *Fine* (F), *Medium* (M), and *Coarse* (C) via per-task quantile binning. In most cases, this mapping corresponds to $w = 0$ for Fine, $w = 1$ for Medium, and $w \geq 2$ for Coarse. We evaluate seven training settings: (1) **Pure F/M/C** ($1 : 0 : 0$): trains exclusively on one granularity class; (2) **Centroid** ($1 : 1 : 1$): uniformly sample from all three classes; (3) **Axial F/M/C** ($3 : 1 : 1$): emphasizes one class with some exposure to the rest classes. Our dataset provides 50 training and 10 evaluation instances per task, each paired with reference trajectories generated by BFWS width-based planner from Lipovetzky and Geffner (2017) (see § E for details).

Action-Decoding Archetypes. The primary architectural difference among latest VLA models lies in their action-decoding strategies (Zhong et al., 2025). Hence, we employ a unified VLM backbone but vary three representative action-decoding strategies (see Figure 2 left panel): (1) **Autoregressive** (AR), exemplified by RT-2 (Zitkovich et al., 2023), OpenVLA (Kim et al., 2024) and others (Reed et al., 2022; Huang et al., 2024; O’Neill et al., 2024) that predict actions sequentially via causal attention; (2) **(Discrete) Diffusion**, which uses diffusion process for action chunk prediction (Black et al., 2025; Bjorck et al., 2025; Shukor et al., 2025). We employ discrete diffusion (Liang et al., 2025) for compatibility with the discrete action space of our testbed; (3) **Parallel Decoding**, utilizing bidirectional attention for single-pass action prediction to improve throughput, as demonstrated by OpenVLA-OFT (Kim et al., 2025), PD-VLA (Song et al., 2025) and other variants (Yin et al., 2025; Wang et al., 2025; Xiao et al., 2025). Details on model hyperparameters, hardwares and training schedules are in § G.

Research Questions. We address the following questions:

(Q1) Granularity Trade-offs. How does instruction granularity impact Success Rate (SR) on within splits?

(Q2) Generalization. Can VLA agents trained on certain granularities generalize to other unseen granularities?

(Q3) Failure Dynamics. When failing, does the agent get stuck (stagnation) or undo achieved progress (regression)?

5.2 Main Results

We first answer (Q1), short answers as follows:

(A1) VLA performance follows a non-monotonic U-shape, peaking at the Fine and Coarse extremes.

To establish a performance baseline, we first evaluate the models under matched granularity, where training and testing occur on the same instruction class ($F \rightarrow F$, $M \rightarrow M$, $C \rightarrow C$). As shown in Figure 3, all three action-decoding archetypes demonstrate a non-monotonic U-shaped discovery. Note that all instructions are generated via a unified template-based translator that directly verbalizes symbolic rule sets, ensuring uniform vocabulary, syntax, and phrasing across granularity levels and isolating rule width as the causal factor. We interpret this U-shaped curve as follows. When instructions provide a step-by-step mapping to actions (i.e., $w = 0$), the learning burden is minimal, as the agent can directly follow instructions with little reasoning or planning effort. At the coarse end ($w \geq 2$), agents are motivated to develop internal planning skills to ground coarse instructions into actions. Importantly, since each coarse instruction maps to a broader set of visual and action patterns, it helps develop more robust and generalizable language grounding by providing more diverse training data. In contrast, medium instructions ($w = 1$) occupy a sub-optimal grounding regime. They are too sparse to provide a direct language-to-action mapping, yet not coarse enough to foster internal planning skills, thereby resulting in the performance decline.

From a width-based planning perspective, the U-shaped pattern surprises because coarser instructions should monotonically increase planning demands, making learning progressively harder. Yet, a common confounding factor is instruction sequence length (i.e., the number of rule calls required to complete a task), which also impacts SR due to error accumulation. Note that our evaluation uses the same underlying task set across granularities, and thus the reference atomic action sequence length is held constant. The only entangled factor is thus the rule call count. As shown in Figure 5, coarser instructions require fewer rule calls to accomplish the same task, as each instruction maps to a longer action sequence. The medium granularity operates at a moderate number of rule calls. This decouples instruction sequence length from instruction width and confirms the U-shape cannot be attributed to medium-granularity tasks requiring excessively longer instruction sequences or rule calls.

To further decouple the effects of instruction

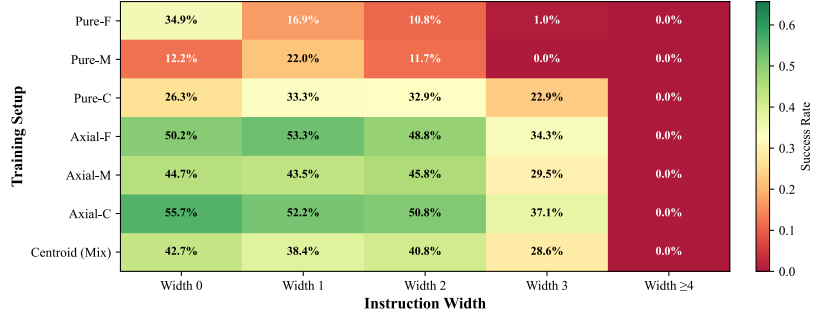
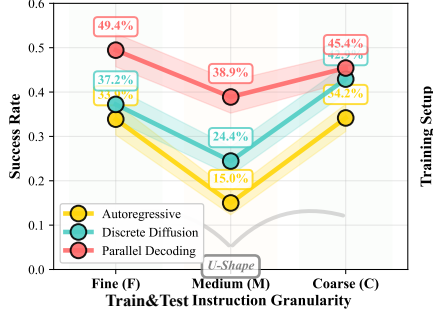


Figure 3: In-Distribution (matched Figure 4: The x-axis are test instruction width, and the y-axis are training train/test) SR across granularities (i.e., setup. Transfer is asymmetric: train coarse $\rightarrow$ test fine works, but train fine train Fine $\rightarrow$ test Fine, etc). A non- $\rightarrow$ test coarse does not. Mixed-granularity training improves generalization. monotonic U-shaped trend appears. *Axial-C* training setting performs best overall.

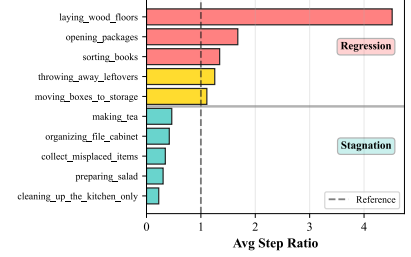
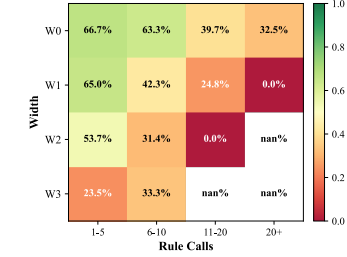
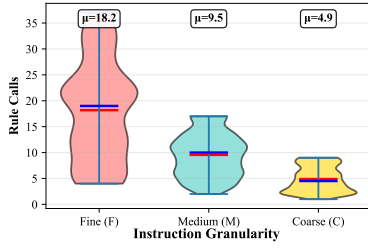


Figure 5: Rule call distribution across instruction granularities. # of rule calls decreases with coarser instructions. It decouples instruction length from width.

Figure 6: Impact of width and horizon on SR. SR decays with task horizon; larger instruction width imposes an additional burden even when horizon is fixed.

Figure 7: Low-width but long task horizon (e.g., laying woods) tends to cause regression, while high-width tasks tend to stagnate. These contribute to the failure of planning.

width and sequence length on VLA performance, we analyze Figure 6, obtained by *Centroid* setting as it has balanced exposure to all granularities. The heatmap provides empirical evidence that, for a fixed number of rule calls, SR decreases as instruction width increases. This demonstrates that the width metric, originally developed for symbolic planning, is a meaningful proxy for the difficulty faced by learning-based VLA agents.

Next, we evaluate cross-granularity generalization, where models are trained on one instruction granularity and evaluated on another. Results are summarized in below:

(A2) Language grounding is asymmetric: models trained on coarse texts generalize to detailed ones, but not vice versa.

Figure 4 reveals a clear asymmetry: models trained on coarse instructions generalize well to fine ones, demonstrating robust language grounding that abstracts away from granularity. Conversely, models trained on fine instructions do not generalize to coarse ones, implying a form of grounding that overfits to step-by-step specification. Mixed-granularity training mitigates this

issue, with the coarse-weighted *Axial-C* variant yielding the best generalization overall.

For (Q3), the failure mode is dependent on task structure:

(A3) Low-width but long-horizon tasks tend to cause regression, while high-width tasks tend to stagnate.

We interpret these failure modes through the lens of width-based planning (see Figure 7). In high-regression tasks, which often require manipulating multiple objects toward a collective goal, agents frequently exhibit subgoal serialization failures. In these settings, the model fails to maintain satisfied fluents over time, cycling back to earlier states. Conversely, stagnation in high-width tasks (e.g., making tea) reflects a width-planning bottleneck: as simultaneous feature dependencies increase ($w \geq 4$), agents struggle to track multiple simultaneous feature dependencies to enable a valid state progression. More analysis is in § H.

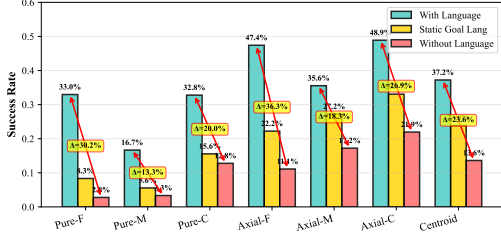


Figure 8: Both mixed and coarse-only training “generalize” better in novel instruction settings (smaller SR gap).

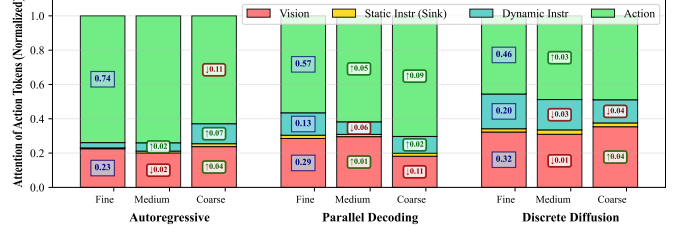


Figure 9: Attention from action tokens. Self-attention dominates, followed by vision; No clear trend across granularity levels.

6 Dilemma of Coarse Instruction Training

Our analyses in (Q1) and (Q3) suggest a practical advantage for training with coarse instructions: models maintain strong in-distribution performance while generalizing effectively to unseen, finer-grained instructions. To further evaluate this generalization capability, we test robustness to more extreme instruction variations: providing only a static goal description (fixed for the entire episode, akin to the LIBERO testbed (Liu et al., 2023)) and providing no instruction at all.

Results are shown in Figure 8. Mixed-granularity training, particularly the *Axial-C* variant, achieves the highest SR when presented with the unseen static-goal descriptions. More strikingly, when the language input is entirely absent, *Axial-C* maintains a non-trivial SR of 21%. This raises a fundamental question: does the success of coarse instruction training stem from improved generalization via robust language grounding, or from “language ignoring”, where the policy becomes invariant to language inputs?

Quantifying Language Ignoring. “Language ignoring” is a phenomenon recently identified in VLA training, where agents appear to over-rely on visual patterns, making their policies invariant to language input (Xing et al., 2025; Fei et al., 2025). A standard metric for this is $\Delta\text{SR} = \text{SR}_{w/\text{lang}} - \text{SR}_{w/o \text{ lang}}$, where a smaller gap indicates stronger ignoring. Our data reveals a trend: models trained purely on coarse instructions (*Pure-C*) exhibit a smaller ΔSR (20.0%) than those trained on fine instructions (*Pure-F*), suggesting coarser training may induce greater language ignoring.

Attention Analysis. To probe the model’s internal reliance on language more directly, we analyze the attention distribution from action tokens to the

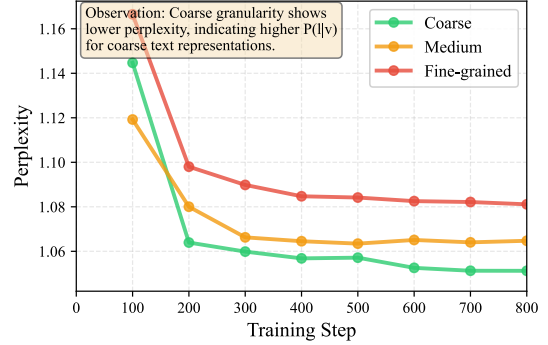


Figure 10: Perplexity of a VLM model trained to predict instructions from visual context, compared across granularities. Coarse instructions are more predictable, supporting high $P(l | v)$ claim.

vision, language, and self-modalities (see Figure 9). Contrary to a simple visual-shortcut hypothesis, we find no clear trend of decreasing attention to the language modality as instruction granularity coarsens. Across all training regimes, self-attention receives the largest share, followed by vision, with language consistently receiving the smallest portion. Notably, autoregressive models attend least to language, which could explain their lower overall SR in Figure 3.

Why Coarse Instructions Encourage Language Ignoring? Xu et al. (2025) posit a causal mechanism: if the language instruction l is highly predictable from visual context v (i.e., large $P(l | v)$), a multimodal model may learn to bypass language processing to save computational energy. This aligns with our benchmark, where expert trajectories are fixed but described at varying granularities: consider a surjective mapping f from a fine-grained label space $L_{\text{fine}} = l_1, \dots, l_n$ to a coarser one $L_{\text{coarse}} = c_1, \dots, c_m$ with $m < n$. The conditional probability for a coarse label aggregates over its constituent fine labels: $P(c_j | v) = \sum_{l_i \in f^{-1}(c_j)} P(l_i | v)$. Since the same total probability mass is distributed over fewer coarse categories,

the average $P(c_j|v)$ is higher. According to the theory, this increased predictability could encourage language ignoring for coarsely-trained models.

Mixed-granularity training mitigates this effect by increasing the diversity of the language space L seen during training, which lowers the average $P(l | v)$. We thus see all mixed-granularity models exhibit larger ΔSR gaps than their pure counterparts, indicating reduced language ignoring. Among these, the fine-weighted mixture *Axial-F* emerges as a particularly balanced choice, achieving a success rate comparable to *Axial-C* while maintaining a substantially larger ΔSR .

Our analysis provides a complementary perspective on the language ignoring phenomenon, demonstrating its variation across instruction granularities, a factor not explored in prior work. The trade-off between final performance and language reliance highlights an open challenge in VLA design: how to structure training for both robust generalization and faithful language grounding.

7 Conclusion and Future Work

This work establishes rule width (w) as a cross-task formalism for instruction granularity, moving beyond syntactic heuristics to quantify the intrinsic planning burden of language guidance. Unlike syntactic heuristics (e.g., counting mentioned entities), w captures the intrinsic planning demand imposed by environment dynamics, enabling consistent comparison across diverse embodied tasks. Our controlled analysis reveals a non-monotonic U-shaped relationship between w and performance. This granularity-centric diagnosis demonstrates that deliberate instruction design and mixed-granularity training are essential for robust VLA alignment.

Future Work. Several promising directions follow. First, while this work focuses on instruction granularity, a natural extension is to investigate representation granularity across other modalities. Visual perception itself can be processed at varying levels of abstraction (e.g., semantic segmentations or structured scene graphs). Future research could explore how the granularity of visual inputs interacts with instruction width to shape the agent’s internal planning demand. Second, the feature set Φ used to define width should not be limited to predefined predicates. Developing learnable, hierarchical feature modules that dynamically construct task-relevant state abstractions would allow the model to adjust its own “internal granularity”.

Investigating the cross-modal alignment between language, vision, and feature granularities will be crucial for future VLA studies.

References

- Arjun R. Akula, Spandana Gella, Aishwarya Padmakumar, Mahdi Namazifar, Mohit Bansal, Jesse Thomason, and Dilek Hakkani-Tur. 2022. ALFRED-L: investigating the role of language for action learning in interactive visual environments. In *EMNLP*, pages 9369–9378. Association for Computational Linguistics.
- Johan Bjorck, Fernando Castañeda, Nikita Cherniadev, Xingye Da, Runyu Ding, Linxi Fan, Yu Fang, Dieter Fox, Fengyuan Hu, Spencer Huang, and 1 others. 2025. Gr00t n1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*.
- Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Robert Equi, Chelsea Finn, Niccolo Fusai, Manuel Y. Galliker, Dibya Ghosh, Lachy Groom, Karol Hausman, brian ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Devin LeBlanc, Sergey Levine, and 16 others. 2025. $\pi_{0.5}$: a vision-language-action model with open-world generalization. In *Proceedings of The 9th Conference on Robot Learning*, volume 305 of *Proceedings of Machine Learning Research*, pages 17–40. PMLR.
- Blai Bonet, Guillem Francès, and Hector Geffner. 2019. Learning features and abstract actions for computing generalized plans. In *AAAI*, pages 2703–2710. AAAI Press.
- Blai Bonet and Hector Geffner. 2021. General policies, representations, and planning width. In *AAAI*, pages 11764–11773. AAAI Press.
- Li Dong and Mirella Lapata. 2018. Coarse-to-fine decoding for neural semantic parsing. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 731–742, Melbourne, Australia. Association for Computational Linguistics.
- Dominik Drexler, Jendrik Seipp, and Hector Geffner. 2022. Learning sketches for decomposing planning problems into subproblems of bounded width. In *ICAPS*, pages 62–70. AAAI Press.
- Dominik Drexler, Jendrik Seipp, and Hector Geffner. 2024. Expressing and exploiting subgoal structure in classical planning using sketches. *J. Artif. Intell. Res.*, 80.
- Senyu Fei, Siyin Wang, Junhao Shi, Zihao Dai, Jikun Cai, Pengfang Qian, Li Ji, Xinzhe He, Shiduo Zhang, Zhaoye Fei, Jinlan Fu, Jingjing Gong, and Xipeng

729	Qiu. 2025. Libero-plus: In-depth robustness analysis of vision-language-action models . <i>Preprint</i> , arXiv:2510.13626.	783
730		784
731		785
732	Catherine Glossop, William Chen, Arjun Bhorkar, Dhruv Shah, and Sergey Levine. 2025. Cast: Counterfactual labels improve instruction following in vision-language-action models . <i>Preprint</i> , arXiv:2508.13446.	786
733		787
734		788
735		789
736		790
737	Danijar Hafner. 2021. Benchmarking the spectrum of agent capabilities. <i>arXiv preprint arXiv:2109.06780</i> .	791
738		792
739	Chi-Pin Huang, Yueh-Hua Wu, Min-Hung Chen, Yu-Chiang Frank Wang, and Fu-En Yang. 2025a. Thinkact: Vision-language-action reasoning via reinforced visual latent planning . <i>Preprint</i> , arXiv:2507.16815.	793
740		794
741		795
742		796
743		797
744	Hui Huang, Jiaheng Liu, Yancheng He, Shilong Li, Bing Xu, Conghui Zhu, Muyun Yang, and Tiejun Zhao. 2025b. Musc: Improving complex instruction following with multi-granularity self-contrastive training. In <i>ACL (1)</i> , pages 10667–10686. Association for Computational Linguistics.	798
745		799
746		800
747		801
748		802
749		803
750	Jiangyong Huang, Silong Yong, Xiaojian Ma, Xiongkun Linghu, Puhao Li, Yan Wang, Qing Li, Song-Chun Zhu, Baoxiong Jia, and Siyuan Huang. 2024. An embodied generalist agent in 3d world. In <i>ICML</i> .	804
751		805
752		806
753		807
754	Emily Jin, Jiaheng Hu, Zhuoyi Huang, Ruohan Zhang, Jiajun Wu, Li Fei-Fei, and Roberto Martín-Martín. 2023. Mini-BEHAVIOR: A procedurally generated benchmark for long-horizon decision-making in embodied AI . In <i>NeurIPS 2023 Workshop on Generalization in Planning</i> .	808
755		809
756		810
757		811
758		812
759		813
760	Siddharth Karamcheti, Suraj Nair, Ashwin Balakrishna, Percy Liang, Thomas Kollar, and Dorsa Sadigh. 2024. Prismatic vlms: Investigating the design space of visually-conditioned language models . In <i>Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024</i> .	814
761		815
762		816
763		817
764		818
765		819
766	Moo Jin Kim, Chelsea Finn, and Percy Liang. 2025. Fine-tuning vision-language-action models: Optimizing speed and success. <i>arXiv preprint arXiv:2502.19645</i> .	820
767		821
768		822
769		823
770	Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Paul Foster, Pannag R. Sanketi, Quan Vuong, Thomas Kollar, Benjamin Burchfiel, Russ Tedrake, Dorsa Sadigh, Sergey Levine, Percy Liang, and Chelsea Finn. 2024. Openvla: An open-source vision-language-action model. In <i>CoRL</i> , volume 270 of <i>Proceedings of Machine Learning Research</i> , pages 2679–2713. PMLR.	824
771		825
772		826
773		827
774		828
775		829
776		830
777		831
778		832
779	Royi Lachmy, Valentina Pyatkin, Avshalom Manevich, and Reut Tsarfaty. 2022. Draw me a flower: Processing and grounding abstraction in natural language. <i>Trans. Assoc. Comput. Linguistics</i> , 10:1341–1356.	833
780		834
781		835
782		836
	Chengshu Li, Fei Xia, Roberto Martín-Martín, Michael Lingelbach, Sanjana Srivastava, Bokui Shen, Kent Elliott Vainio, Cem Gokmen, Gokul Dharan, Tanish Jain, Andrey Kurenkov, Karen Liu, Hyowon Gweon, Jiajun Wu, Li Fei-Fei, and Silvio Savarese. 2022a. igibson 2.0: Object-centric simulation for robot learning of everyday household tasks . In <i>Proceedings of the 5th Conference on Robot Learning</i> , volume 164 of <i>Proceedings of Machine Learning Research</i> , pages 455–465. PMLR.	837
		838
		839
	Chengshu Li, Ruohan Zhang, Josiah Wong, Cem Gokmen, Sanjana Srivastava, Roberto Martín-Martín, Chen Wang, Gabriel Levine, Michael Lingelbach, Jiankai Sun, Mona Anvari, Minjune Hwang, Manasi Sharma, Arman Aydin, Dhruva Bansal, Samuel Hunter, Kyu-Young Kim, Alan Lou, Caleb R. Matthews, and 9 others. 2022b. BEHAVIOR-1K: A benchmark for embodied AI with 1,000 everyday activities and realistic simulation. In <i>CoRL</i> , volume 205 of <i>Proceedings of Machine Learning Research</i> , pages 80–93. PMLR.	840
		841
	Wei Li, Renshan Zhang, Rui Shao, Jie He, and Liqiang Nie. 2025. Cogvla: Cognition-aligned vision-language-action model via instruction-driven routing & sparsification. <i>Advances in neural information processing systems</i> .	842
		843
	Xuanlin Li, Kyle Hsu, Jiayuan Gu, Oier Mees, Karl Pertsch, Homer Rich Walke, Chuyuan Fu, Ishikaa Lunawat, Isabel Sieh, Sean Kirmani, Sergey Levine, Jiajun Wu, Chelsea Finn, Hao Su, Quan Vuong, and Ted Xiao. 2024. Evaluating real-world robot manipulation policies in simulation. In <i>CoRL</i> , volume 270 of <i>Proceedings of Machine Learning Research</i> , pages 3705–3728. PMLR.	844
		845
	Zechang Li, Yuxuan Lai, Yansong Feng, and Dongyan Zhao. 2020. Domain adaptation for semantic parsing . In <i>Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence, IJCAI 2020</i> , pages 3723–3729. ijcai.org.	846
		847
	Zhixuan Liang, Yizhuo Li, Tianshuo Yang, Chengyue Wu, Sitong Mao, Liua Pei, Xiaokang Yang, Jiangmiao Pang, Yao Mu, and Ping Luo. 2025. Discrete diffusion vla: Bringing discrete diffusion to action decoding in vision-language-action policies. <i>arXiv preprint arXiv:2508.20072</i> .	848
		849
	Nir Lipovetzky and Hector Geffner. 2012. Width and serialization of classical planning problems. In <i>ECAI</i> , volume 242 of <i>Frontiers in Artificial Intelligence and Applications</i> , pages 540–545. IOS Press.	850
		851
	Nir Lipovetzky and Hector Geffner. 2017. Best-first width search: Exploration and exploitation in classical planning. In <i>AAAI</i> , pages 3590–3596. AAAI Press.	852
		853
	Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. 2023. LIBERO: benchmarking knowledge transfer for lifelong robot learning. In <i>NeurIPS</i> .	854
		855

840	Yueen Ma, Zixing Song, Yuzheng Zhuang, Jianye Hao, and Irwin King. 2025. A survey on vision-language-action models for embodied ai . <i>Preprint</i> , arXiv:2405.14093.	897
841		898
842		899
843		900
844	Vivek Myers, Chunyuan Zheng, Oier Mees, Kuan Fang, and Sergey Levine. 2024. Policy adaptation via language optimization: Decomposing tasks for few-shot imitation. In <i>CoRL</i> , volume 270 of <i>Proceedings of Machine Learning Research</i> , pages 1402–1426. PMLR.	901
845		902
846		
847		903
848		904
849		
850	Abby O’Neill, Abdul Rehman, Abhiram Maddukuri, Abhishek Gupta, Abhishek Padalkar, Abraham Lee, Acorn Pooley, Agrim Gupta, Ajay Mandlekar, Ajinkya Jain, Albert Tung, Alex Bewley, Alexander Herzog, Alex Irpan, Alexander Khazatsky, Anant Rai, Anchit Gupta, Andrew E. Wang, Anikait Singh, and 260 others. 2024. Open x-embodiment: Robotic learning datasets and RT-X models : Open x-embodiment collaboration. In <i>ICRA</i> , pages 6892–6903. IEEE.	905
851		906
852		907
853		908
854		
855		909
856		910
857		
858		911
859		912
860	Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy V. Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, Mido Assran, Nicolas Ballas, Wojciech Galuba, Russell Howes, Po-Yao Huang, Shang-Wen Li, Ishan Misra, Michael Rabbat, Vasu Sharma, and 7 others. 2024. Dinov2: Learning robust visual features without supervision. <i>Trans. Mach. Learn. Res.</i> , 2024.	913
861		914
862		915
863		
864		916
865		917
866		918
867		919
868		920
869	Scott E. Reed, Konrad Zolna, Emilio Parisotto, Sergio Gómez Colmenarejo, Alexander Novikov, Gabriel Barth-Maron, Mai Gimenez, Yury Sulsky, Jackie Kay, Jost Tobias Springenberg, Tom Eccles, Jake Bruce, Ali Razavi, Ashley Edwards, Nicolas Heess, Yutian Chen, Raia Hadsell, Oriol Vinyals, Mahyar Bordbar, and Nando de Freitas. 2022. A generalist agent. <i>Trans. Mach. Learn. Res.</i> , 2022.	921
870		922
871		923
872		924
873		925
874		926
875		
876		927
877	Lucy Xiaoyang Shi, Brian Ichter, Michael Robert Equi, Liyiming Ke, Karl Pertsch, Quan Vuong, James Tanner, Anna Walling, Haohuan Wang, Niccolo Fusai, Adrian Li-Bell, Danny Driess, Lachy Groom, Sergey Levine, and Chelsea Finn. 2025. Hi robot: Open-ended instruction following with hierarchical vision-language-action models. In <i>ICML</i> .	928
878		929
879		930
880		931
881		
882		932
883		933
884	Mohit Shridhar, Jesse Thomason, Daniel Gordon, Yonatan Bisk, Winson Han, Roozbeh Mottaghi, Luke Zettlemoyer, and Dieter Fox. 2020. Alfred: A benchmark for interpreting grounded instructions for everyday tasks. In <i>Proceedings of the IEEE/CVF conference on computer vision and pattern recognition</i> , pages 10740–10749.	934
885		935
886		936
887		
888		937
889		938
890		939
891	Mustafa Shukor, Dana Aubakirova, Francesco Capuano, Pepijn Kooijmans, Steven Palma, Adil Zouitine, Michel Aractingi, Caroline Pascal, Martino Russi, Andres Marafioti, and 1 others. 2025. Smolvla: A vision-language-action model for affordable and efficient robotics. <i>arXiv preprint arXiv:2506.01844</i> .	940
892		941
893		
894		942
895		943
896		944
		945
		946
		947
		948
		949

Yifan Zhong, Fengshuo Bai, Shaofei Cai, Xuchuan Huang, Zhang Chen, Xiaowei Zhang, Yuanfei Wang, Shaoyang Guo, Tianrui Guan, Ka Nam Lui, and 1 others. 2025. A survey on vision-language-action models: An action tokenization perspective. *arXiv preprint arXiv:2507.01925*.

Brianna Zitkovich, Tianhe Yu, Sichun Xu, Peng Xu, Ted Xiao, Fei Xia, Jialin Wu, Paul Wohlhart, Stefan Welker, Ayzaan Wahid, Quan Vuong, Vincent Vanhoucke, Huong T. Tran, Radu Soricut, Anikait Singh, Jaspiar Singh, Pierre Sermanet, Pannag R. Sanketi, Grecia Salazar, and 35 others. 2023. RT-2: vision-language-action models transfer web knowledge to robotic control. In *CoRL*, volume 229 of *Proceedings of Machine Learning Research*, pages 2165–2183. PMLR.

A Technical Appendix

Appendix Table of Contents

1. **Anticipatory defense on the choice of test environments and metrics (§ B)**
Justification for using MINI-BEHAVIOR over alternatives (e.g., iGibson, CALVIN) based on controllability, symbolic abstraction, and isolation of language-grounding challenges.
2. **Symbolic Representation of the Embodied Task (§ C)** Formal definition of states, actions, and planning problems using classical STRIPS/PDDL semantics.
3. **Details of the MINI-BEHAVIOR Environment (§ D)** Overview of scene layout, object types, action space, and perception model.
4. **Details of the MINI-BEHAVIOR-GRAN Extension (§ E)** Procedure for generating multi-granularity instructions, details on the feature set, oracle instructor, and dataset construction.
5. **Rule Set and Natural Language Prompt Examples (§ F)** Details of the rule set bank of 20 tasks and sample generated instructions at different granularities.
6. **Implementation Details (§ G)** Model architecture (Prismatic-7B backbone, SigLIP+DINOv2 visual encoders), training hyperparameters, optimizer settings.
7. **Additional Experimental Results (§ H)** Extended analysis and ablations.

B Anticipatory defense on the choice of test environments and metrics

In this work, we selected Mini-BEHAVIOR as our testbed. This choice is motivated by several key considerations aligned with our research objectives. Our research objective is not to solve the full stack of robotic embodiment (which includes perception, continuous control, and physics), but specifically to isolate and analyze the impact of instruction granularity on language grounding and long-horizon planning.

Therefore, high-fidelity simulators (e.g., iGibson (Li et al., 2022a), BEHAVIOR-1K (Li et al., 2022b)) introduce significant noise through complex texture rendering and continuous motor control. While realistic, these factors act as confounders. When an agent fails in such environments, it is often difficult to attribute the failure to a lack of linguistic understanding (granularity issues) or a failure in low-level execution (e.g., grasping physics or occlusion). Mini-BEHAVIOR retains the high-level decision complexity of household tasks—long horizons, multi-object interactions, and state dependencies—while abstracting away low-level sensorimotor noise.

In the perspective of fast prototyping and iterative experimentation, we have to also consider the rendering and physics simulation costs. Mini-BEHAVIOR’s grid-based discrete control and simplified visual representation allow for rapid data generation and model training.

Another consideration is the diversity of the task structures so that we can systematically vary instruction granularity across a wide range of scenarios. As such, Crafter (Hafner, 2021), which focuses on survival in a single domain, lacks the diverse task structures needed to obtain robust insights about instruction granularity.

Table 2 provides a detailed comparison of Mini-BEHAVIOR against other prominent benchmarks, highlighting why they were less suitable for this specific investigation.

C Symbolic Representation of the Embodied Task

We formalize the Mini-BEHAVIOR environment as a deterministic grid-based symbolic world. This allows us to represent the challenge as a classical planning problem $\mathcal{P} = \langle \mathcal{F}, \mathcal{A}, s_0, G \rangle$, where:

- $\mathcal{F}$ is a finite set of propositional fluents representing the state variables of the environment (e.g.,

Table 2: Comparison of Embodied AI Environments regarding their suitability for studying Instruction Granularity.

Environment	Control	Horizon	Visuals	Symbolic Abstraction	Limitation
Mini-BEHAVIOR(*)	Discrete	Long	Grid	Support	Ideal balance: Supports symbolic state abstraction; diverse task suite; removes control noise while retaining logic complexity.
Crafter	Discrete	Long	Cartoon	Partial	Single task suite (survival); partially observable (memory heavy); few object types.
Meta-World	Continuous	Short	3D	No	Continuous control; no symbolic abstraction; focus on multi-task RL/manipulation, not long-horizon planning.
LIBERO	Continuous	Short	3D	No	Robotic arm focus; continuous control; no symbolic abstraction; VLA success rate already saturated (tasks relatively easy).
RoboTwin 2.0	Continuous	Short	3D	No	Continuous control; no symbolic abstraction; dual-arm manipulation focus; hard to segment expert trajectories and construct granular instructions.
CALVIN	Continuous	Medium	3D	No	Hard to design instruction granularity due to continuous action space (though provides diverse language); confounds planning with control difficulties.
ALFRED	Discrete	Long	3D Photo	Support	3D partially observable; poor infrastructure support for VLA training; high rendering cost.
iGibson / BEHAVIOR-1K	Continuous	Long	3D Photo	Partial	Good for long-horizon tasks, but 3D realistic vision context becomes a confounder

object locations, states like *cooked* or *open*).

- $\mathcal{A}$ is a finite set of actions available to the agent.
- $s_0 \subseteq \mathcal{F}$ is the initial state configuration.
- $G \subseteq \mathcal{F}$ is the goal condition that must be satisfied.

A state $s \in \mathcal{S}$ is defined as a truth valuation over $\mathcal{F}$, with the state space $\mathcal{S} \subseteq 2^{\mathcal{F}}$ encompassing all valid combinations of fluent truth values.

D Details of MINI-BEHAVIOR Environment

To operationalize our formalism, we developed a **Universal PDDL Domain Model** (available at [minibehavior_gran_domain_model.pddl](#) in the codebase) capable of supporting all 20 household tasks without modification. This unified model comprises over 1,000 lines of PDDL 2.1 definitions and is fully compatible with the LAMA planner. We highly recommend checking the codebase for the complete domain model.

Table 3: Planning complexity in typical tasks.

Metric	Typical Range
Objects	15–40
Grid Locations	20–60
Ground Actions	10^3 – 10^5
Plan Length	50–200 primitive actions
Planning Time (LAMA)	0.5–300+ seconds

Search Space Characteristics

E Details of MINI-BEHAVIOR-GRAN Extension

MINI-BEHAVIOR provides the foundational task skeletons, which we extend in MINI-BEHAVIOR-GRAN to support multi-granularity instruction generation. A core component of this extension is the definition of a symbolic feature set used to generate rule-based policy sketches.

E.1 Feature Categories

The environment defines two primary categories of features: Boolean State Features and Quantitative Features.

E.1.1 Boolean State Features

These features evaluate whether a specific object instance satisfies a predicate. They require grounding (binding to specific object instances, e.g., rag-01).

E.1.2 Quantitative and Spatial Features

These features capture distances, counts, and spatial relationships. Crucially, they support temporal comparison allowing conditions such as “distance decreased” or “count dropped to zero.”

- **DistanceToNearest(type, condition):** Measures the Manhattan distance from the agent to the nearest object of a specific type.

– *Conditions:* $=0$ (adjacent), >0 , increase, decrease, change.

- **Count(type, condition, function):** Counts the number of objects of a specific type that satisfy a filter function.

– *Conditions:* $=0$, >0 , increase, decrease.

E.2 Count Functions

A diverse set of counting functions is implemented to support complex logical conditions (e.g., “all shoes are clean” is equivalent to *count_not_cleaned* == 0).

E.3 Reference Plan Generation and Task Complexity Analysis

While the original MINI-BEHAVIOR framework provides the high-level skeletons for household activities, it does not provide reference plans by default. Thus, we utilize the BFWS (Best-First Width Search) classical planner to generate reference plans for all 20 tasks in MINI-BEHAVIOR-GRAN. These reference plans serve as the ground truth for our agents and allow us to quantitatively categorize the complexity of each task.

Detailed statistics regarding the specific rule sets used for instruction generation and the resulting natural language statistics are discussed in the next section.

F Rule Set and Natural Language Prompt Examples For Different Tasks

We present the width-0 rule set for 10 over 20 household tasks in the Mini-BEHAVIOR-Gran environment. For $w > 0$, please refer to the codebase at [mini-behavior-gran/mini_behavior_utils/policy_sketch/](https://github.com/mini-behavior-gran/mini_behavior_utils/policy_sketch/).

F.1 Task 1: Laying Wood Floors

Features:

- H : holding an isolated wood (plywood) 1124
- p : distance to the nearest isolated wood 1125
- t : distance to an arbitrary (non-held) wood we’ll place next to 1126
- n : number of isolated woods 1128

$\{\neg H, p > 0, n > 0\} \mapsto \{p \downarrow, t?\}$; move close to isolated wood
 $\{\neg H, p = 0, n > 0\} \mapsto \{H\}$; pick up isolated wood when reachable 1129
 $\{H, t > 0, n > 0\} \mapsto \{t \downarrow\}$; move to some wood location
 $\{H, n > 0, t = 0\} \mapsto \{H?, n \downarrow, p?\}$; drop the isolated wood next to other wood

F.2 Task 2: Preparing Salad

Features:

- H_n : holding a not-sliceable food 1132
- H_s : holding a sliceable food 1133
- H_k : holding a slicer 1134
- U : number of sliceable food that is not sliced yet 1135
- d_p : distance to the selected incomplete plate p 1136
- d_n, d_s : distances to nearest not-sliceable / sliceable 1137
- k : distance to a slicer (knife) 1138
- B : selected plate p complete bottom salad 1139
- n : number of incomplete plates 1140
- nn : number of plates that do not have bottom salad yet 1141

$\{U > 0, \neg H_k, k > 0\} \mapsto \{k \downarrow\}$; move to slicer
 $\{U > 0, \neg H_k, k = 0\} \mapsto \{H_k\}$; hold a slicer
Acquire slicer: $\{U > 0, H_k, d_s > 0\} \mapsto \{d_s \downarrow\}$; move to sliceable food 1145
 $\{U > 0, H_k, d_s = 0\} \mapsto \{U \downarrow, \neg H_k\}$; cut sliceable food
 $\{U = 0, H_k\} \mapsto \{\neg H_k\}$; put down slicer
 $\{n > 0, U = 0, nn > 0, \neg B, \neg H_n, d_n > 0\} \mapsto \{d_n \downarrow\}$
Place bottom salad: $\{n > 0, U = 0, nn > 0, \neg B, \neg H_n, d_n = 0\} \mapsto \{H_n\}$ 1146
 $\{n > 0, U = 0, nn > 0, \neg B, H_n, d_p > 0\} \mapsto \{d_p \downarrow\}$
 $\{n > 0, U = 0, nn > 0, \neg B, H_n, d_p = 0\} \mapsto \{B, nn \downarrow, \neg H_n\}$
 $\{nn = 0, n > 0, U = 0, \neg H_s, d_s > 0\} \mapsto \{d_s \downarrow\}$; move to sliceable food
Handle sliceable salad: $\{nn = 0, n > 0, U = 0, \neg H_s, d_s = 0\} \mapsto \{H_s\}$; pick up sliceable food 1147
 $\{nn = 0, n > 0, U = 0, H_s, d_p > 0\} \mapsto \{d_p \downarrow\}$; move to the plate
 $\{nn = 0, n > 0, U = 0, H_s, d_p = 0\} \mapsto \{n \downarrow, \neg H_s\}$; put down sliced food

F.3 Task 3: Cleaning Up the Kitchen Only

Features:

- **Holding flags:** H_b (blender), H_f (food), H_s (soap), H_r (rag), H_p (plate), H_o (vegetable oil) 1150
- **Distances:** d_{ct} (countertop), d_{fr} (fridge), d_{si} (sink), d_{c1} (cabinet for plate), d_{c2} (cabinet for oil), d_x (to object x) 1151
- **State booleans:** $\text{Open}(c)$, $\text{Tog}(s)$, Soaked , $\text{DustFree}(c)$, $\text{Clean}(p)$, $\text{OilIn}(c_2)$, Diff 1152
- **Counters:** N_b (blenders not on countertop), N_a (apples not in fridge), N_{cas} (casseroles not in fridge), N_s (soaps not next to sink), 1153

Table 4: Boolean State Features

Feature	Applicable Objects	Semantics
Holding	All Items	Is the agent holding the specified object?
isOpened	Cabinets, Drawers, Doors	Is the container/door open?
isToggled	Sinks, Stoves, Switches	Is the appliance turned on?
isSoaked	Rags, Sponges	Is the cleaning tool wet?
isCleaned	Shoes, Dishes, Floor	Is the object free of stains?
isDustFree	Furniture, Surfaces	Is the object free of dust?

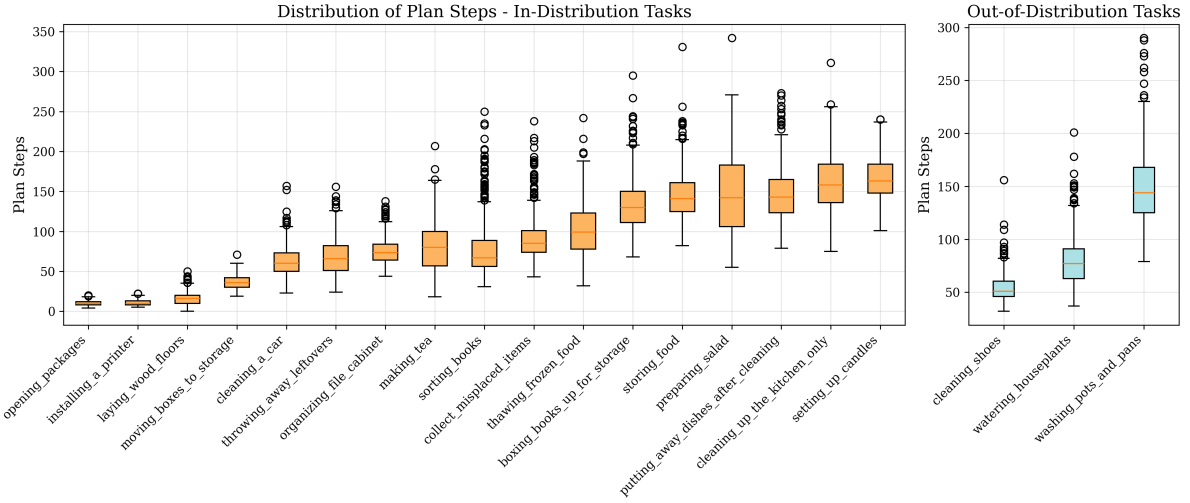


Figure 11: Distribution of reference plan steps required for tasks in MINI-BEHAVIOR-GRAN obtained by BWFS classical planner.

Table 5: Available Count Functions for Rule Definitions

Category	Function Description
State-Based	count_not_cleaned: Objects that are currently stained. count_not_soaked: Cleaning tools that are dry. count_not_toggled: Appliances that are turned off. count_closed: Containers/doors that are closed. count_not_sliced: Food items that require slicing.
Spatial	count_not_onfloor: Objects not placed on the floor layer. count_isolated: Objects with no neighbors of the same type. count_not_inside_target: Objects not contained within a specific furniture. count_not_ontop: Objects not placed on top of a specific surface. count_not_near_target: Objects not within 1 step of a target type.
Task-Specific	count_incomplete_salad: Plates lacking necessary ingredients. count_not_complete_tables: Tables lacking specific setups (e.g., candles).

N_{cab} (cabinets not dusted), N_r (rags not next to sink), N_{p_clean} (plates not clean), N_{oil} (oil not in cabinet), N_{p_store} (plates not stored), N_{close} (closed furniture)

1161 N_{cab} (cabinets not dusted), N_r (rags not next to sink), N_{p_clean} (plates not clean), N_{oil} (oil not in cabinet), N_{p_store} (plates not stored), N_{close} (closed furniture)

1162

1163

1164

1165

1166

1167

Open Cabinet and Fridge: $\{N_{close} > 0, d_c > 0\} \mapsto \{d_c \downarrow\}$; move toward closed furniture

$\{N_{close} > 0, d_c = 0\} \mapsto \{N_{close} \downarrow\}$

$\{N_{close} = 0, \neg H_o, d_{oil} > 0\} \mapsto \{d_{oil} \downarrow\}$; approach oil

$\{N_{close} = 0, \neg H_o, d_{oil} = 0\} \mapsto \{H_o\}$; pick up oil

Oil placement: $\{N_{close} = 0, H_o, N_{oil} > 0, d_{c2} > 0\} \mapsto \{d_{c2} \downarrow\}$; go to cabinet c_2

$\{N_{close} = 0, H_o, N_{oil} > 0, d_{c2} = 0\} \mapsto \{N_{oil} \downarrow, \neg H_o\}$; place oil in c_2

$\{\neg \text{Tog}(s), d_{si} > 0\} \mapsto \{d_{si} \downarrow\}$; toggle sink on

$\{\neg \text{Tog}(s), d_{si} = 0\} \mapsto \{\text{Tog}(s)\}$

$\{\neg H_r, d_{rag} > 0\} \mapsto \{d_{rag} \downarrow\}$; get rag

$\{\neg H_r, d_{rag} = 0\} \mapsto \{H_r\}$

Plate cleaning: $\{H_r, \neg \text{Soaked}, d_{si} > 0\} \mapsto \{d_{si} \downarrow\}$; soak rag

$\{H_r, \neg \text{Soaked}, d_{si} = 0\} \mapsto \{\text{Soaked}\}$

$\{H_r, \text{Soaked}, N_{p_clean} > 0, d_{plate} > 0\} \mapsto \{d_{plate} \downarrow\}$

$\{H_r, \text{Soaked}, N_{p_clean} > 0, d_{plate} = 0\} \mapsto \{N_{p_clean} \downarrow, d_{plate}?\}$

$\{N_{close} = 0, N_{oil} = 0, N_{p_clean} = 0, \neg H_p, N_{p_store} > 0, d_{plate} > 0\} \mapsto \{d_{plate} \downarrow\}$

Plate placement distinct from oil: $\{N_{close} = 0, N_{oil} = 0, N_{p_clean} = 0, \neg H_p, N_{p_store} > 0, d_{plate} = 0\} \mapsto \{H_p\}$

$\{N_{close} = 0, N_{oil} = 0, N_{p_clean} = 0, H_p, N_{p_store} > 0, d_{c1} > 0\} \mapsto \{d_{c1} \downarrow\}$

$\{N_{close} = 0, N_{oil} = 0, N_{p_clean} = 0, H_p, N_{p_store} > 0, d_{c1} = 0\} \mapsto \{N_{p_store} \downarrow, \neg H_p\}$

$\{N_b > 0, \neg H_b, d_{blender} > 0\} \mapsto \{d_{blender} \downarrow\}$

Blender to countertop: $\{N_b > 0, \neg H_b, d_{blender} = 0\} \mapsto \{H_b\}$

$\{H_b, N_b > 0, d_{ct} > 0\} \mapsto \{d_{ct} \downarrow\}$

$\{H_b, N_b > 0, d_{ct} = 0\} \mapsto \{N_b \downarrow, \neg H_b\}$

$\{N_{close} = 0, \neg H_f, d_{apple} > 0\} \mapsto \{d_{apple} \downarrow\}$

Apple to fridge: $\{N_{close} = 0, \neg H_f, d_{apple} = 0\} \mapsto \{H_f\}$

$\{N_{close} = 0, H_f, N_a > 0, d_{fr} > 0\} \mapsto \{d_{fr} \downarrow\}$

$\{N_{close} = 0, H_f, N_a > 0, d_{fr} = 0\} \mapsto \{N_a \downarrow, \neg H_f\}$

$\{N_{close} = 0, \neg H_f, d_{casserole} > 0\} \mapsto \{d_{casserole} \downarrow\}$

Casserole to fridge: $\{N_{close} = 0, \neg H_f, d_{casserole} = 0\} \mapsto \{H_f\}$

$\{N_{close} = 0, H_f, N_{cas} > 0, d_{fr} > 0\} \mapsto \{d_{fr} \downarrow\}$

$\{N_{close} = 0, H_f, N_{cas} > 0, d_{fr} = 0\} \mapsto \{N_{cas} \downarrow, \neg H_f\}$

$\{\neg H_r, d_{rag} > 0\} \mapsto \{d_{rag} \downarrow\}$; get rag

$\{\neg H_r, d_{rag} = 0\} \mapsto \{H_r\}$

Cabinet dusting: $\{H_r, N_{cab} > 0, d_c > 0\} \mapsto \{d_c \downarrow\}$; clean cabinet

$\{H_r, N_{cab} > 0, d_c = 0\} \mapsto \{N_{cab} \downarrow, \neg H_r\}$

$\{N_{cab} = 0, N_{p_clean} = 0, \neg H_r, d_{rag} > 0\} \mapsto \{d_{rag} \downarrow\}$

$\{N_{cab} = 0, N_{p_clean} = 0, \neg H_r, d_{rag} = 0\} \mapsto \{H_r\}$

Rag near/inside sink: $\{N_{cab} = 0, N_{p_clean} = 0, H_r, N_r > 0, d_{si} > 0\} \mapsto \{d_{si} \downarrow\}$

$\{N_{cab} = 0, N_{p_clean} = 0, H_r, N_r > 0, d_{si} = 0\} \mapsto \{N_r \downarrow, \neg H_r\}$

$\{N_{cab} = 0, N_{p_clean} = 0, \neg H_s, d_{soap} > 0\} \mapsto \{d_{soap} \downarrow\}$

$\{N_{cab} = 0, N_{p_clean} = 0, \neg H_s, d_{soap} = 0\} \mapsto \{H_s\}$

Soap next to sink: $\{N_{cab} = 0, N_{p_clean} = 0, H_s, N_s > 0, d_{si} > 0\} \mapsto \{d_{si} \downarrow\}$

$\{N_{cab} = 0, N_{p_clean} = 0, H_s, N_s > 0, d_{si} = 0\} \mapsto \{N_s \downarrow, \neg H_s\}$

F.4 Task 4: Organizing File Cabinet

Features:

Task	Split	Class	Goal Description	Mean Steps
opening_packages	ID	short	Open every package.	10.2
installing_a_printer	ID	short	Place a printer on a table and turn it on.	10.6
laying_wood_floors	ID	short	Lay every plywood sheet on the floor, each touching at least one other sheet.	15.9
moving_boxes_to_storage	ID	short	Place every carton inside a shelf.	36.3
cleaning_a_car	ID	short	Make at least one car dust-free. Place a rag and soap together inside a bucket.	62.6
throwing_away_leftovers	ID	middle	Throw away every hamburger by placing it in a trash can.	68.3
organizing_file_cabinet	ID	middle	Put a marker on a table and file every folder and document in a cabinet.	75.5
making_tea	ID	middle	Slice a lemon. Put a teapot on an activated stove. Keep a soaked teabag with the teapot.	78.7
sorting_books	ID	middle	Place every book and every hardback on a shelf.	78.8
collect_misplaced_items	ID	middle	Place all gym shoes, necklaces, notebooks, and socks on a table.	90.7
thawing_frozen_food	ID	middle	Place a date label next to a fish. Put every fish next to a sink. Set an olive next to a sink.	101.7
boxing_books_up_for_storage	ID	long	Place every book inside a box.	134.4
storing_food	ID	long	Store every cookable item inside a cabinet.	144.6
preparing_salad	ID	long	Slice all apples and tomatoes; place them on plates. Put every radish and lettuce on plates. Ensure each plate holds at least one cookable item.	146.7
putting_away_dishes_after_cleaning	ID	long	Put every plate inside a cabinet.	147.7
cleaning_up_the_kitchen_only	ID	long	Put every blender on a countertop. Refrigerate all apples and casseroles. Keep soap next to a sink. Make plates unstained and cabinets dust-free. Place each rag either beside or inside the sink. Store plates and vegetable oil in different cabinets.	161.0
setting_up_candles	ID	long	On every table, place three distinct candles on top.	165.2
cleaning_shoes	OOD	short	Leave a towel on the floor and make sure every shoe is unstained and dust-free.	54.2
watering_houseplants	OOD	middle	Water every potted plant until it is soaked.	79.4
washing_pots_and_pans	OOD	long	Clean every teapot, kettle, and pan so they're unstained, then store each inside a cabinet.	149.5

Table 6: Classification of tasks in MINI-BEHAVIOR-GRAN based on the median number of plan steps, as determined by the BWFS planner. Tasks are categorized into short, middle, and long classes to reflect varying planning complexity based on plan length.

- H_m : holding a marker
- H_f : holding a folder
- H_d : holding a document
- p_m : distance to the nearest marker
- p_f : distance to the nearest folder
- p_d : distance to the nearest document
- p_t : distance to a table
- p_c : distance to a valid cabinet target
- N_m : number of markers not on a table
- N_f : number of folders not in a cabinet
- N_d : number of documents not in a cabinet
- N_o : number of closed cabinets

Opening Cabinet: $\{N_o > 0, p_c > 0\} \mapsto \{p_c \downarrow\}$; move toward a closed cabinet
 $\{N_o > 0, p_c = 0\} \mapsto \{N_o \downarrow\}$; open the cabinet when reachable
Marker task: $\{N_m > 0, \neg H_m, p_m > 0\} \mapsto \{p_m \downarrow, p_t?\}$; move toward the nearest marker
 $\{N_m > 0, \neg H_m, p_m = 0\} \mapsto \{H_m, p_t?\}$; pick up the marker when reachable
 $\{N_m > 0, H_m, p_t > 0\} \mapsto \{p_t \downarrow\}$; move toward a table
 $\{N_m > 0, H_m, p_t = 0\} \mapsto \{N_m \downarrow, \neg H_m, p_m?\}$; put marker on table
Folder task: $\{N_f > 0, N_o = 0, \neg H_f, p_f > 0\} \mapsto \{p_f \downarrow, p_c?\}$
 $\{N_f > 0, N_o = 0, H_f, p_c > 0\} \mapsto \{p_c \downarrow\}$
 $\{N_f > 0, N_o = 0, H_f, p_c = 0\} \mapsto \{N_f \downarrow, \neg H_f, p_f?\}$
 $\{N_d > 0, N_o = 0, \neg H_d, p_d > 0\} \mapsto \{p_d \downarrow, p_c?\}$
Document task: $\{N_d > 0, N_o = 0, \neg H_d, p_d = 0\} \mapsto \{H_d, p_c?\}$
 $\{N_d > 0, N_o = 0, H_d, p_c > 0\} \mapsto \{p_c \downarrow\}$
 $\{N_d > 0, N_o = 0, H_d, p_c = 0\} \mapsto \{N_d \downarrow, \neg H_d, p_d?\}$

F.5 Task 5: Thawing Frozen Food

Features:

- H_{fish} : holding a fish
- H_{olive} : holding an olive
- H_{date} : holding a date
- p_{fish} : distance to the nearest fish
- p_{olive} : distance to the nearest olive
- p_{date} : distance to the nearest date
- p_{sink} : distance to a sink
- N_{fish} : number of fish not next to a sink
- N_{olive} : number of olives not next to a sink
- N_{date} : number of dates not next to a fish

Fish Task: $\{\neg H_{fish}, p_{fish} > 0\} \mapsto \{p_{fish} \downarrow\}$
 $\{\neg H_{fish}, p_{fish} = 0\} \mapsto \{H_{fish}\}$
 $\{H_{fish}, p_{sink} > 0\} \mapsto \{p_{sink} \downarrow\}$
 $\{H_{fish}, p_{sink} = 0\} \mapsto \{N_{fish} \downarrow, \neg H_{fish}, p_{fish}?\}$
Olive Task: $\{\neg H_{olive}, p_{olive} > 0\} \mapsto \{p_{olive} \downarrow\}$
 $\{\neg H_{olive}, p_{olive} = 0\} \mapsto \{H_{olive}\}$
 $\{H_{olive}, p_{sink} > 0\} \mapsto \{p_{sink} \downarrow\}$
 $\{H_{olive}, p_{sink} = 0\} \mapsto \{N_{olive} \downarrow, \neg H_{olive}, p_{olive}?\}$
Date Task: $\{\neg H_{date}, N_{fish\ olive} = 0, p_{date} > 0\} \mapsto \{p_{date} \downarrow\}$
 $\{\neg H_{date}, N_{fish\ olive} = 0, p_{date} = 0\} \mapsto \{H_{date}\}$
 $\{H_{date}, N_{fish\ olive} = 0, p_{fish} > 0\} \mapsto \{p_{fish} \downarrow\}$
 $\{H_{date}, N_{fish\ olive} = 0, p_{fish} = 0\} \mapsto \{N_{date} \downarrow, \neg H_{date}, p_{date}?\}$

F.6 Task 6: Making Tea

Features:

- H_l : holding a lemon
- H_{knife} : holding a knife
- H_t : holding a teapot
- H_{tb} : holding a tea bag
- p_l : distance to the nearest lemon
- p_{knife} : distance to the knife
- p_t : distance to the nearest teapot
- p_{tb} : distance to the nearest tea bag
- p_s : distance to the stove
- p_{sink} : distance to the sink
- $\text{is_sliced}(\text{lemon})$: whether the lemon is sliced
- $\text{is_toggled}(\text{stove})$: whether the stove is toggled on
- $\text{is_toggled}(\text{sink})$: whether the sink is toggled on
- $\text{is_soaked}(\text{tea_bag})$: whether the tea bag is soaked
- $\text{atsamelocation}(\text{tea_bag}, \text{teapot})$: tea bag at same location as teapot
- $\text{onTop}(\text{teapot}, \text{stove})$: teapot on stove

Toggle stove: $\{\neg \text{is_toggled}(\text{stove}), p_s > 0\} \mapsto \{p_s \downarrow\}$
 $\{\neg \text{is_toggled}(\text{stove}), p_s = 0\} \mapsto \{\text{is_toggled}(\text{stove})\}$
 $\{\neg H_{knife}, p_{knife} > 0, \neg \text{is_sliced}(\text{lemon})\} \mapsto \{p_{knife} \downarrow\}$
 $\{\neg H_{knife}, p_{knife} = 0, \neg \text{is_sliced}(\text{lemon})\} \mapsto \{H_{knife}\}$
Slice lemon: $\{H_{knife}, \neg \text{is_sliced}(\text{lemon}), p_l > 0\} \mapsto \{p_l \downarrow\}$
 $\{H_{knife}, \neg \text{is_sliced}(\text{lemon}), p_l = 0\} \mapsto \{\text{is_sliced}(\text{lemon})\}$
 $\{H_{knife}, \text{is_sliced}(\text{lemon})\} \mapsto \{\neg H_{knife}, p_{knife}?\}$
 $\{\neg \text{atsameloc}(\text{tb}, \text{tp}), \neg \text{onTop}(\text{tp}, \text{st}), \neg H_t, p_t > 0\} \mapsto \{p_t \downarrow\}$
 $\{\neg \text{atsameloc}(\text{tb}, \text{tp}), \neg \text{onTop}(\text{tp}, \text{st}), \neg H_t, p_t = 0\} \mapsto \{H_t\}$
 $\{\neg \text{atsameloc}(\text{tb}, \text{tp}), \neg \text{onTop}(\text{tp}, \text{st}), H_t, p_s > 0\} \mapsto \{p_s \downarrow\}$
Handle teapot: $\{\neg \text{atsameloc}(\text{tb}, \text{tp}), \neg \text{onTop}(\text{tp}, \text{st}), H_t, p_s = 0\} \mapsto \{\text{onTop}(\text{tp}, \text{st}), \neg H_t, p_t?\}$
 $\{\neg \text{atsameloc}(\text{tb}, \text{tp}), \text{onTop}(\text{tp}, \text{st}), \neg H_{tb}, p_{tb} > 0\} \mapsto \{p_{tb} \downarrow\}$
 $\{\neg \text{atsameloc}(\text{tb}, \text{tp}), \text{onTop}(\text{tp}, \text{st}), \neg H_{tb}, p_{tb} = 0\} \mapsto \{H_{tb}\}$
 $\{\neg \text{atsameloc}(\text{tb}, \text{tp}), \text{onTop}(\text{tp}, \text{st}), H_{tb}, p_t > 0\} \mapsto \{p_t \downarrow\}$
 $\{H_{tb}, \neg \text{atsameloc}(\text{tb}, \text{tp}), \text{onTop}(\text{tp}, \text{st}), p_t = 0\} \mapsto \{\text{atsameloc}(\text{tb}, \text{tp}), \neg H_{tb}, p_{tb}?\}$

F.7 Task 7: Opening Packages

Features:

- p : distance to the nearest package
- N : number of unopened packages

$\{N > 0, p > 0\} \mapsto \{p \downarrow\}$; move toward the nearest unopened package
 $\{N > 0, p = 0\} \mapsto \{N \downarrow\}$; open the package when reachable

F.8 Task 8: Boxing Books Up for Storage

Features:

- H : holding an unboxed book
- p : distance to the nearest unboxed book
- b : distance to a valid box target
- N : number of unboxed books

$\{\neg H, p > 0\} \mapsto \{p \downarrow, b?\}$; move toward the nearest unboxed book
 $\{\neg H, p = 0\} \mapsto \{H\}$; pick up the book when reachable
 $\{H, N > 0, b > 0\} \mapsto \{b \downarrow\}$; move toward a chosen box spot
 $\{H, N > 0, b = 0\} \mapsto \{\neg H, N \downarrow, p?\}$; place the book into the box

F.9 Task 9: Collect Misplaced Items

Features:

- H_s : holding a gym shoe
- H_n : holding a necklace
- H_b : holding a notebook
- H_k : holding a sock
- p_s : distance to the nearest gym shoe
- p_n : distance to the nearest necklace
- p_b : distance to the nearest notebook
- p_k : distance to the nearest sock
- p_t : distance to a table
- N_s : number of gym shoes not on a table
- N_n : number of necklaces not on a table
- N_b : number of notebooks not on a table
- N_k : number of socks not on a table

$\{\neg H_s, N_s > 0, p_s > 0\} \mapsto \{p_s \downarrow, p_t?\}$
Gym shoe task: $\{\neg H_s, N_s > 0, p_s = 0\} \mapsto \{H_s, p_t?\}$
 $\{H_s, N_s > 0, p_t > 0\} \mapsto \{p_t \downarrow\}$
 $\{H_s, N_s > 0, p_t = 0\} \mapsto \{N_s \downarrow, \neg H_s, p_s?\}$
 $\{\neg H_n, N_n > 0, p_n > 0\} \mapsto \{p_n \downarrow, p_t?\}$
Necklace task: $\{\neg H_n, N_n > 0, p_n = 0\} \mapsto \{H_n, p_t?\}$
 $\{H_n, N_n > 0, p_t > 0\} \mapsto \{p_t \downarrow\}$
 $\{H_n, N_n > 0, p_t = 0\} \mapsto \{N_n \downarrow, \neg H_n, p_n?\}$
 $\{\neg H_b, N_b > 0, p_b > 0\} \mapsto \{p_b \downarrow, p_t?\}$
Notebook task: $\{\neg H_b, N_b > 0, p_b = 0\} \mapsto \{H_b, p_t?\}$
 $\{H_b, N_b > 0, p_t > 0\} \mapsto \{p_t \downarrow\}$
 $\{H_b, N_b > 0, p_t = 0\} \mapsto \{N_b \downarrow, \neg H_b, p_b?\}$
 $\{\neg H_k, N_k > 0, p_k > 0\} \mapsto \{p_k \downarrow, p_t?\}$
Sock task: $\{\neg H_k, N_k > 0, p_k = 0\} \mapsto \{H_k, p_t?\}$
 $\{H_k, N_k > 0, p_t > 0\} \mapsto \{p_t \downarrow\}$
 $\{H_k, N_k > 0, p_t = 0\} \mapsto \{N_k \downarrow, \neg H_k, p_k?\}$

F.10 Task 10: Putting Away Dishes After Cleaning

Features:

- H : holding a plate
- p : distance to the nearest plate
- b : distance to a valid cabinet target
- N : number of unboxed plates
- is_opened : whether the cabinet is opened

$\{\neg \text{is_opened}, b > 0\} \mapsto \{b \downarrow\}$; move toward the cabinet
 $\{\neg \text{is_opened}, b = 0\} \mapsto \{\text{is_opened}\}$; open the cabinet when reachable
 $\{\neg H, N > 0, p > 0, \text{is_opened}\} \mapsto \{p \downarrow, b?\}$
 $\{\neg H, N > 0, p = 0, \text{is_opened}\} \mapsto \{H, b?\}$
 $\{H, N > 0, b > 0, \text{is_opened}\} \mapsto \{b \downarrow\}$
 $\{H, N > 0, b = 0, \text{is_opened}\} \mapsto \{\neg H, N \downarrow, p?\}$

F.11 Instruction Generation Statistics

We used a template-based generator to translate the symbolic rule sets into natural language instructions, ensuring alignment with the VLA's training

data format. Table 7 presents the summary statistics for the generated instructions across the three granularity levels.

G Implementation Details

G.1 Model Architecture

Our policy is built upon the OpenVLA architecture (Kim et al., 2024). Specifically, we utilize the Prismatic-7B backbone (Karamcheti et al., 2024), which integrates a frozen Llama-2-7B language model with fused visual features. The visual encoder consists of a dual-stream setup combining SigLIP (for semantic understanding) and DINOv2 (for fine-grained spatial geometry) (Zhai et al., 2023; Oquab et al., 2024).

G.2 Training Configuration

We fine-tune the model using Low-Rank Adaptation (LoRA) on the language model backbone, keeping the visual encoders frozen. All training setting variants (Pure, Centroid, Axial) share the same underlying training hyperparameters to ensure fair comparison.

Hardware and Duration: Training was conducted on a single node equipped with $8 \times$ NVIDIA A100 GPUs. A standard training run for 100,000 steps took approximately 23 hours to complete.

Table 8 summarizes the core hyperparameters used across our experiments.

Table 8: Hyperparameters for OpenVLA fine-tuning on MINI-BEHAVIOR-GRAN.

Hyperparameter	Value
Base Model	Prismatic-7B (Llama-2)
Visual Encoders	SigLIP + DINOv2
Parameter Efficient Tuning	LoRA (Rank $r = 32$)
Compute Hardware	$8 \times$ NVIDIA A100
Training Duration	~ 23 hours
Optimization Steps	100,000
Steps Before Decay	80,000
Learning Rate	2.5×10^{-4}
Batch Size (per GPU)	7 – 8

H Additional Experimental Results

This section provides additional detailed per-task performance breakdowns, and failure mode distributions.

H.1 Detailed Task Performance

Figure 12 details the success rates across all 20 tasks in the benchmark. We see some tasks are

too difficult to solve and thus low across all instruction granularity. And then we also see the U shape trends in most tasks, showing the same phenomenon we discussed in (Q1) that both very fine and very coarse instructions are easier for VLA to follow.

H.2 Failure Analysis

Figure 13 presents the distribution of failure types across varying instruction widths. It confirms that low width encounters regression failures more frequently, while high width predominantly leads to stagnation failures. This aligns with our earlier observations in (Q3) regarding the distinct failure modes associated with different instruction granularities.

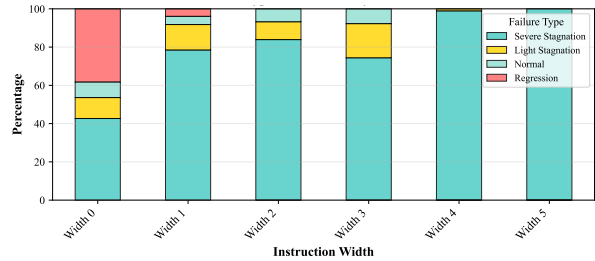


Figure 13: Failure type distribution across instruction widths.

H.3 Sample Generated Instructions by Granularity

To illustrate the qualitative differences between granularity levels, we provide five random samples for each category below. Note how Low (L) granularity explicitly specifies navigation and pre-conditions (e.g., “move toward,” “not yet at”), while High (H) granularity focuses almost exclusively on the final state change (e.g., “reduce the count of...”), shown in Table 9.

Table 7: Statistics of generated natural language instructions across granularity levels.

Granularity	Count	Avg Length (words)	Std Length	Min	Max	Unique Tokens	TTR (Type-Token Ratio)	Avg Sent. Length
Fine (F)	1636	41.12	11.36	19	87	140	0.0021	41.12
Medium (M)	859	38.62	12.76	22	81	108	0.0033	38.62
Coarse (C)	587	33.60	13.39	22	95	94	0.0048	33.60

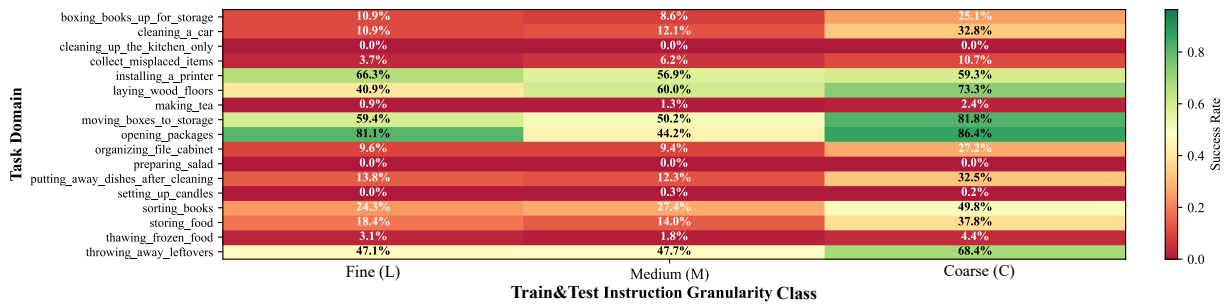


Figure 12: Detailed per-task success rates.

Fine Granularity (F) Samples

1. At step 11: When the count of uncleaned plate is above 0, not yet at sink 0 and sink is off, please try to move toward sink 0.
2. At step 8: When not yet at soap 0, not holding soap 0, the count of unwiped car is 0 and the count of rag not inside bucket is 0, please try to move toward soap 0.
3. At step 27: When the count of chip and oatmeal sugar and vegetable oil not inside cabinet is above 0, you are at oatmeal 1 and not holding oatmeal 1, please pick up oatmeal 1.
4. At step 15: When the count of plate not inside cabinet is above 0, you are at plate 3, the count of closed cabinet is 0 and not holding plate 3, please pick up plate 3.
5. At step 13: When the count of fish and olive not near sink is above 0, not yet at olive 0 and not holding olive 0, please try to move toward olive 0.

Medium Granularity (M) Samples

1. At step 2: When the count of plate not inside cabinet is above 0, the count of closed cabinet is 0 and not holding plate 2, please pick up plate 2.
2. At step 6: When the count of uncleaned plate is above 0, the count of dry rag is above 0 and holding rag 0, please reduce the count of dry rag.
3. At step 9: When the count of plate not inside cabinet 0 is above 0, the count of closed cabinet is 0, the count of vegetable oil not inside cabinet 1 is 0, the count of uncleaned plate is 0 and not holding plate 0, please pick up plate 0.
4. At step 13: When the count of book not inside box is above 0 and not holding book 2, please pick up book 2.
5. At step 9: When the count of chip and oatmeal sugar and vegetable oil not inside cabinet is above 0, holding chip 1 and the count of closed cabinet is 0, please not holding chip 1, then reduce the count of chip and oatmeal sugar and vegetable oil not inside cabinet.

Coarse Granularity (C) Samples

1. At step 2: When the count of hamburger not inside ashcan is above 0, please reduce the count of hamburger not inside ashcan.
2. At step 5: When the count of plate not inside cabinet is above 0 and the count of closed cabinet is 0, please reduce the count of plate not inside cabinet.
3. At step 3: When the count of teapot not on stove is above 0, please reduce the count of teapot not on stove.
4. At step 9: When the count of plate not inside cabinet is above 0 and the count of closed cabinet is 0, please reduce the count of plate not inside cabinet.
5. At step 8: When the count of chip and oatmeal sugar and vegetable oil not inside cabinet is above 0 and the count of closed cabinet is 0, please reduce the count of chip and oatmeal sugar and vegetable oil not inside cabinet.

Table 9: Sample generated instructions across granularity levels.